Auditing an income-prediction model for sex bias

Supervised Learning & Model Engineering (MSA-DATB09-04) — Project report (draft)

Manel Rahali, Soumaya Atoui, Hadiqa Javaid

Albert School, Fall 2026

Repository: https://github.com/mrahali-cell/Supervised-Learning-Model-Engineering

Dataset: Option 1 — Adult / Census Income

`fetch_openml(name="adult", version=2)`, 48,842 people, 15 columns, 1994 US Census [1].

Abstract

A company sells models that predict whether a person earns more than $50K a year; its clients use the predictions for decisions such as audits. We perform the *model-audit step*: build the best honest model the data allows, then measure how unequally it treats women and men, reduce part of that gap and show what it costs. This draft covers the first stages: problem framing, exploratory analysis, preprocessing and feature engineering, class imbalance, the model progression, and the tuning of logistic regression and the random forest. With six legitimate inputs, logistic regression, a random forest and LightGBM (default settings) reach the same level (about 0.71 PR-AUC, 0.89 ROC-AUC). For the random forest, validation curves show that limiting tree growth fixes its overfitting, and a search space built from these curves (tuning v2) keeps the ranking of the first tuning while restoring trustworthy probabilities (Brier 0.110 instead of 0.136). The EDA shows that the inputs are proxies for sex, which the later stages (calibration, explainability, fairness audit, test-set evaluation) will examine.

1 Problem framing and evaluation protocol

Use case. If an income model treats some groups worse than others, the audit decisions built on it become unfair, and its clients face sanctions, legal action and financial loss. Our deliverable is the evidence an auditor needs: how biased the model is by sex, one mitigation, and what it costs. The research question is *how biased by sex is a model that predicts income $>$ \$50K without ever seeing sex?* Our hypothesis is that it rebuilds the gender gap through proxies: marital status, occupation and hours worked.

Target and metrics. The target `high_income` = 1 if income $>$ \$50K (23.9% of people). In an audit neither error dominates, so we chose the orientation that makes the metrics informative: the rare class as positive (Session 6). Accuracy is uninformative (always answering "no" scores 76%), and ROC-AUC rewards the many correct rejections of low earners. We therefore rank models with **PR-AUC**, whose floor is the prevalence (0.239), report ROC-AUC beside it, and judge the probabilities with the **Brier score**. With the first orientation (low income = 1) the PR-AUC floor was 0.76 and every model scored about 0.96, which hid the differences.

Fairness definition, fixed before any result. Primary: **equalized odds** (equal true- and false-positive rates for women and men) [4], because for a general-purpose model both kinds of error can harm someone. Legal screen: the **four-fifths rule** on selection rates. Demographic parity and predictive parity will also be reported (Section 9).

Honest protocol. After fixed cleaning rules, the data is split once into a development set (80%, 39,032 people) and a test set (20%, 9,758 people), stratified on the target (23.94% high earners in both) with `random_state=42`. All model choices use 5-fold stratified cross-validation on the development set; the test set stays locked until the final evaluation. Anything that learns from the data (imputation, scaling, encoding) lives inside the Pipeline and is re-fitted in every fold (Session 5). Table 1 maps the project to the course and the code.

#	Step	Where in the code	Req.	Status
1–3	Framing, EDA, cleaning, stratified split	`notebooks/01_eda.ipynb`, `src/data.py`	1	done
4	Feature engineering + preprocessing in one Pipeline	`src/pipeline.py`	2, 5	done
5	Baseline progression: Dummy → LR → RF → LightGBM	`tuning/baseline.py`	3, 4	done
6–7	Diagnosis (validation curves), Optuna tuning, MLflow	`tuning/random_forest.py`, `tuning/tune.py`	6, 7	LR, RF done; LightGBM in progress
8	Calibration	`src/evaluate.py`	9	to do on the final model
9	Explainability (SHAP)		8	to do
10	Fairness audit and mitigation	`src/fairness.py`	10	to do on the final model
11	Test set, once: final metrics, fairness before / after		9, 10	to do

Table 1: The project in 11 steps. Scripts run; notebooks only explain results; `docs/findings_log.md` records every finding with its numbers.

2 Data and exploratory analysis

Integrity. Missing values affect three columns: `workclass` (5.73%), `occupation` (5.75%), `native-country` (1.75%). They are not random: all 2,799 people missing `workclass` also miss `occupation`; the 10 extra missing occupations belong to people who never worked, for whom occupation is *not applicable* (set to "Never-worked"). Other missing values become an explicit "Unknown" category inside the Pipeline, which keeps the information that the value was missing. We removed 52 exact duplicates (0.11%) so the same record cannot sit on both sides of the split. Extreme values are census caps (244 capital gains at 99,999; 137 people at 99 hours; 55 at age 90) and were kept; the IQR rule is unusable when 91.7% of capital gains are zero. Categories with a few dozen people were merged by meaning and income rate: *Without-pay* (21) and *Never-worked* (10) into *No-pay*, 37 *Married-AF-spouse* into *Married-civ-spouse*, 15 *Armed-Forces* into *Protective-serv*; `native-country` was grouped into United States / Mexico / Other. These fixed rules learn nothing from the data and are safe before the split.

Leakage audit. For each column we asked whether it would be known at prediction time (Session 1). Capital gains fail: 98.4% of people with a gain $\geq$ \$10,000 earn more than \$50K, and all 244 at the cap do; the variable is probably derived from income and is not collected by a short questionnaire. We dropped `capital-gain` and `capital-loss`. This is why our scores (about 0.89 ROC-AUC) must not be compared with published ones (0.92–0.93), which use them [2].

Columns kept out. `education` duplicates `education-num`; `fnlwgt` is a sampling weight, not a trait (medians 178,811 vs 176,729 by class); `relationship` reveals sex (all 19,716 Husbands and 2,331 Wives are married). `sex`, `race` and `native-country` are protected characteristics, kept only for the audit. Six inputs remain: `age`, `workclass`, `education-num`, `marital-status`, `occupation`, `hours-per-week`. The strongest signals are marital status (44.6% of married people earn more than \$50K vs 4.5–10.1% for the other statuses), occupation, education (15.9% at level 9, 41.3% at 13, 74.0% at 15), work class and age.

Where the gender gap lives. 30.4% of men vs 10.9% of women earn more than \$50K. Figure 1 shows that among married people the rates are equal (44.8% vs 44.6%), but 60.9% of men are married vs 15.3% of women, and gaps remain within education and hours groups. Occupations differ too (25.3% of women in administrative jobs vs 5.9% of men; 2.2% vs 18.6% in craft and repair), and women work 36.4 hours on

average vs 42.4 for men. Leaving `sex` out therefore does not make a model blind to it: its inputs are proxies ("fairness through unawareness" fails, Session 9). These are descriptive associations, not causes. The split keeps enough people per sex to audit (356 high-earning women in the test set), but two race groups have fewer than 100 people in the test set and can only be described.

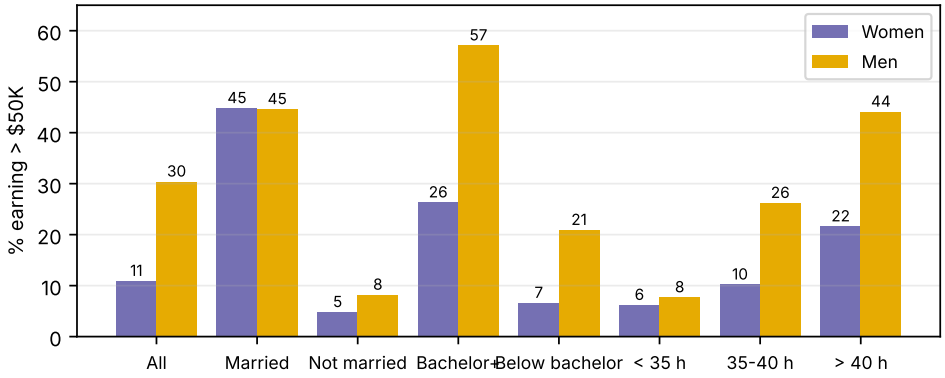


Figure 1: Share earning $>$ \$50K, women vs men, overall and within groups (EDA). Married women and men have the same rate; the overall gap comes largely from who is married, plus gaps within education and hours groups.

3 Feature engineering and the Pipeline

Each engineered feature is a fixed rule justified by an EDA pattern (share earning $\leq$ \$50K in the development set):

- `is_married`: married 55.5% vs everyone else 93.6%;
- `has_degree` (`education-num` ≥ 13, a bachelor's degree or more): 52.0% vs 83.9%;
- `part_time` ($<$ 35 h) and `overtime` ($>$ 40 h): about 93% below 35 h, 78.7% at 40 h, 56–65% above 40 h. Both medians are 40 h, which had hidden this signal in the first EDA;
- `age_squared`: income rises with age, then falls (98.9% at 24 or younger, 60.6% at 45–54, 79.6% at 65+), a U-shape a linear model cannot draw from `age` alone.

The Pipeline (`build_pipeline(model)`) chains a `FunctionTransformer` that adds these features, a `ColumnTransformer` and the model. Numbers are imputed with the training median (the most neutral choice) and standardized; categories get "Unknown" then one-hot encoding with `handle_unknown="ignore"`, so a category absent from a training fold does not crash; the binary features pass through. Six inputs become 37 columns (32 without the engineered features). One-hot encoding avoids inventing an order between categories; scaling matters for the regularized logistic regression and changes nothing for trees. The features lift logistic regression from 0.688 to 0.708 PR-AUC; the tree models find the same patterns by themselves (Table 3).

4 Class imbalance

The positive class is 23.9% of people. We tested class weighting as the course suggests, as a hyperparameter: `class_weight="balanced"` for logistic regression and the random forest, `scale_pos_weight` for LightGBM. Weighting never improved the ranking and always degraded the probabilities (Table 2). Session 6 notes that weights help when a model underfits the rare class; ours do not. We therefore do not re-weight: the imbalance will be handled by the **decision threshold**, chosen at the decision stage, which leaves the probabilities intact **[to do: threshold choice, with the fairness audit]**. We did not use SMOTE: half of our inputs are categorical, a case where interpolating between people is undefined. In the code, weights are part of the Optuna search spaces (`tuning/tune.py`).

Model (5-fold CV, development set)	PR-AUC	ROC-AUC	Brier
Logistic regression, no weights (base features)	0.688	0.883	0.114
Logistic regression, balanced weights (base features)	0.684	0.883	0.143
LightGBM defaults, no weights	0.716	0.892	0.110
LightGBM defaults, balanced weights	0.716	0.892	0.137
Random forest (200 trees, sklearn defaults), no weights	0.642	0.859	0.130
Random forest (200 trees, sklearn defaults), balanced weights	0.628	0.859	0.145

Table 2: Class weights leave the ranking unchanged or worse and degrade the probabilities (higher Brier score).

5 Modelling progression

Table 3 compares the course progression on the same folds and metrics, all with default settings. The fold-to-fold standard deviation of PR-AUC is about 0.013, so smaller differences are not real.

Model (engineered features, default settings)	PR-AUC	ROC-AUC	Brier	Recall at best F1
Dummy (predicts the prior)	0.239	0.500	0.182	–
Logistic regression	0.708 ± 0.014	0.889 ± 0.006	0.111	0.762
Random forest (300 trees, leaf 5)	0.711 ± 0.013	0.890 ± 0.005	0.111	0.787
LightGBM [5]	0.716 ± 0.012	0.892 ± 0.005	0.110	0.778

Table 3: Model progression, 5-fold CV on the development set (mean $\pm$ std over folds). Recall at the best-F1 threshold, pooled out of fold (slightly optimistic, for comparison only).

What each step bought. Logistic regression (Session 2) captures almost all the signal: +0.47 PR-AUC over the Dummy. The random forest (Session 4, bagging) and LightGBM (boosting) add 0.003 and 0.008, within the noise. Three families with very different capacity reaching the same level suggest the limit is the *information* in six inputs, not the model. The progression is still useful: it rules out that a more flexible model behaves very differently, and it keeps a transparent benchmark (logistic regression, whose coefficients expose the proxies) next to the stronger models.

6 Hyperparameter tuning and experiment tracking

Tuning v1. One Optuna study per model [6] (TPE sampler, seed 42, 25 trials each), objective = mean 5-fold PR-AUC of the complete Pipeline, so every trial is cross-validated and the test set is never used. Class weighting was part of each search space. Logistic regression reached 0.708 ± 0.014 (C = 5.18, L2 penalty, no class weights), no gain over its default. The random forest reached 0.715 ± 0.014 (300 trees, depth 15, leaf 10, `log2` columns, `balanced_subsample` weights), +0.004 over its baseline but with a worse Brier score (0.136 vs 0.111). The best trial is selected on the same folds it is scored on, so these scores are slightly optimistic (Session 5).

Diagnosis before tuning (random forest). A random forest does not minimize a global loss: each tree picks, node by node, the split that most reduces impurity (Gini or entropy). One fully grown tree memorizes: 0.977 train vs 0.444 validation PR-AUC. Bagging 100 such trees (sklearn defaults) reaches 0.639 but keeps a 0.33 train–validation gap; requiring 5 people per leaf brings validation to 0.711 with a 0.09 gap. We then drew one validation curve per course hyperparameter, changing one setting while the others stay at sklearn defaults (Figure 2). Growth limits matter most: `min_samples_leaf` peaks at 10 (0.714, plateau 5–20), `max_depth` at 12 (0.713), with depth 4 underfitting (0.684 with a 0.004 gap) and unlimited depth overfitting; `min_samples_split` plateaus at 50–200. Smaller bootstrap samples help (`max_samples` 30%: 0.685 vs 0.642). Gini and entropy are identical (0.642); more than 200 trees changes nothing and never closes the gap; class weights lower PR-AUC and raise the Brier score (0.130 $\rightarrow$ 0.145).

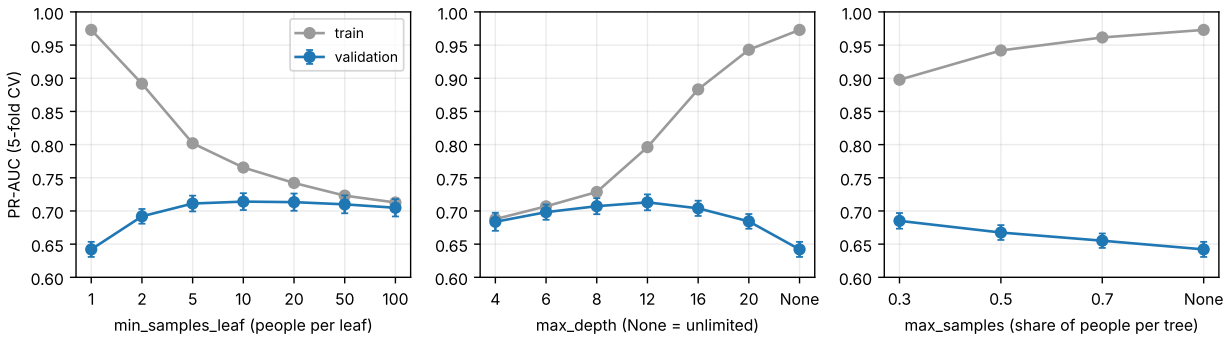


Figure 2: Random forest validation curves (5-fold CV, 200 trees, other settings at sklearn defaults; error bars = fold std). Left: the bias–variance picture, from memorizing (leaf 1) to underfitting (leaf 100). Centre: depth 4 underfits, unlimited depth overfits. Right: smaller samples per tree reduce overfitting.

Tuning v2 (random forest). Each choice of the v2 search space comes from a curve: settings with no or a harmful effect are fixed (300 trees, Gini, no class weights); each searched range is a plateau plus a margin (leaf 5–30, depth 8–20, split 2–200, `max_samples` 30–100%, `max_features` in {sqrt, log2, 0.3}). Objective, budget and seed are those of v1, so the only difference is the search space. A full grid over these settings would need thousands of combinations; Optuna's first 10 trials are random search, then its TPE sampler concentrates on promising regions (Session 5). All 25 trials scored between 0.709 and 0.715: the space lies on the plateau, and the guided trials added 0.0005 over the random ones. The settings interact: alone the best leaf size was 10, but with 39% of people per tree Optuna prefers leaf 6.

Random forest	Train PR-AUC	Val. PR-AUC	Gap	Val. ROC-AUC	Val. Brier
One tree, fully grown	0.977	0.444	0.533	0.734	0.196
sklearn defaults (100 trees, leaf 1)	0.972	0.639	0.333	0.857	0.131
Baseline (300 trees, leaf 5)	0.803	0.711	0.092	0.890	0.111
Tuned v1 (Optuna, 25 trials)	0.749	0.715	0.035	0.891	0.136
Tuned v2 (space from the curves)	0.754	**0.715**	0.039	0.891	**0.110**

Table 4: Random forest, same 5 folds. v2 keeps v1's ranking with a small gap and trustworthy probabilities (best: leaf 6, depth 16, split 10, 39% of people per tree, sqrt columns).

Logistic regression. Tuning v1 is the only tuning so far; no validation-curve diagnosis has been run for it **[to do: Soumaya: logistic regression diagnosis / v2, if done]**.

LightGBM. **[to do: Hadiqa: LightGBM diagnosis and tuning (`tuning/lightgbm.py`).]**

Experiment tracking. Every Optuna study is a parent MLflow run (model, version, budget, objective, CV scheme, best parameters and scores, optimization-history figure) with one nested run per trial (its hyperparameters, mean and standard deviation of PR-AUC and ROC-AUC, Brier score, per-fold metrics). Tuning v1 produced 78 runs (3 parents + 75 trials) and the random forest v2 26 more. Runs are stored in a local SQLite file; the best parameters of each version are versioned in Git (`tuning/results/<version>/`). **[to do: MLflow screenshot, Appendix A]**

7 Calibration

[to do: reliability curve, Brier score and ECE of the final model; over- or under-confident; whether its probabilities can be trusted as-is]

8 Explainability with SHAP

[to do: global importance and local explanations of the final model; do the important features make business sense; proxy check by sex]

9 Fairness and bias audit

[to do: metrics by sex (selection rate, TPR, FPR, precision) against the definitions of Section 1; one mitigation; performance and fairness gaps before / after on the untouched test set]

10 Final evaluation on the test set

[to do: final model, evaluated once on the 9,758 test people]

11 Limitations and conclusion so far

- **Data.** A 1994 US snapshot, with sex recorded as binary and an arbitrary $50K threshold that limits its value as a fairness benchmark [3]. Results describe this population, not today's.
- **Ceiling.** Six legitimate inputs carry about 0.71 PR-AUC of signal; we removed stronger but leaking or sex-revealing columns on purpose. More tuning does not raise this level; only new legitimate information could.
- **Optimism.** Tuned scores are selected on the same folds they are scored on; the test evaluation will give the honest figure.

So far, the model is as good as the six inputs allow: three model families and two tunings of the random forest reach the same level, and the random forest's tuning v2 shows how a search space read from validation curves keeps that ranking with trustworthy probabilities. The EDA shows that the inputs are strong proxies for sex; the next stages test whether the model reproduces the income gap between women and men, and what reducing it costs. **[to do: final conclusion]**

Contribution statement

Member	Contributions
Manel Rahali	GitHub repository and branch workflow; EDA (data integrity, types, outliers, rare categories, features vs target, leakage audit, cleaning and split, preview of the transformed data); `src/data.py`; preprocessing in `src/pipeline.py`; project architecture (`src/` library, `tuning/` scripts); random forest diagnosis, validation curves and tuning v2; README; report coordination.
Soumaya Atoui	Business case and target orientation; feature engineering; EDA steps on groups, proxies and framing; baseline progression, Optuna tuning v1 and MLflow tracking; calibration and fairness scripts; findings log **[to do: confirm]**.
Hadiqa Javaid	LightGBM diagnosis and tuning **[to do: complete]**.

Use of AI coding assistants. We used Claude (Anthropic) as an assistant: to discuss course concepts, to draft and refactor code (project architecture, diagnosis and tuning scripts), and to draft parts of this report from our findings log. Every script was run by the team, every number comes from our runs, and all methodological choices were reviewed by us. **[to do: teammates: add your own use, if any]**

References

[1] R. Kohavi. Scaling up the accuracy of naive-Bayes classifiers: a decision-tree hybrid. *KDD*, 1996 (the Adult dataset).

[2] N. Chakrabarty, S. Biswas. A statistical approach to adult census income level prediction. arXiv:1810.10076, 2018.

[3] F. Ding, M. Hardt, J. Miller, L. Schmidt. Retiring Adult: new datasets for fair machine learning. *NeurIPS*, 2021.

[4] M. Hardt, E. Price, N. Srebro. Equality of opportunity in supervised learning. *NeurIPS*, 2016.

[5] G. Ke et al. LightGBM: a highly efficient gradient boosting decision tree. *NeurIPS*, 2017.

[6] T. Akiba et al. Optuna: a next-generation hyperparameter optimization framework. *KDD*, 2019.

A Experiment tracking

[to do: screenshot of the MLflow runs table (parent run and its 25 trials) and the parallel-coordinates view of the random forest v2 trials]

B Reproducing the results

From the project root, with the environment active: `python3 -m src.data`; `python3 -m src.pipeline`; `python3 -m tuning.baseline`; `python3 -m tuning.random_forest baseline` and `curves`; `python3 -m tuning.tune -version v2 -model rf`. Setup and troubleshooting are in the README.